

Acceso y manipulación del DOM

Selección de elementos

La selección de elementos es el primer paso para interactuar con el DOM. JavaScript proporciona varios métodos para encontrar elementos específicos en una página web.

Métodos de selección básicos

`getElementById()`

Selecciona un elemento por su atributo `id`. Es el método más rápido y directo.

```
// HTML: <div id="mi-contenedor">Contenido</div>
let contenedor = document.getElementById('mi-contenedor');
console.log(contenedor); // <div id="mi-contenedor">Contenido</div>

// Si no existe, devuelve null
let noExiste = document.getElementById('no-existe');
console.log(noExiste); // null
```



`getElementsByTagName()`

Selecciona todos los elementos de una etiqueta específica. Devuelve una `HTMLCollection`.

```
// Obtener todos los párrafos
let parrafos = document.getElementsByTagName('p');
console.log(parrafos.length); // Número de párrafos

// Recorrer todos los párrafos
for (let i = 0; i < parrafos.length; i++) {
```



```
    console.log(parrafos[i].textContent);  
}
```

getElementsByClassName()

Selecciona elementos por su clase CSS. También devuelve una HTMLCollection.

```
// HTML: <p class="destacado">Texto 1</p>  
//       <div class="destacado">Texto 2</div>  
let destacados = document.getElementsByClassName('destacado');  
console.log(destacados.length); // 2  
  
// Acceder al primer elemento  
if (destacados.length > 0) {  
    console.log(destacados[0].textContent);  
}
```



getElementsByName()

Selecciona elementos por su atributo `name` (común en formularios).

```
// HTML: <input name="email" type="text">  
//       <input name="email" type="hidden">  
let emails = document.getElementsByName('email');  
console.log(emails.length); // 2
```



Métodos de selección modernos (más potentes) #

querySelector()

Selecciona el **primer elemento** que coincida con un selector CSS.

```
// Por ID  
let elemento = document.querySelector('#mi-id');  
  
// Por clase  
let elemento = document.querySelector('.mi-clase');
```



```
// Por etiqueta
let elemento = document.querySelector('p');

// Selectores complejos
let elemento = document.querySelector('div.contenedor > p.destacado');
let elemento = document.querySelector('input[type="email"]');
let elemento = document.querySelector('li:first-child');
```

querySelectorAll()

Selecciona **todos los elementos** que coincidan con un selector CSS. Devuelve un NodeList.

```
// Todos los elementos con clase 'item'
let items = document.querySelectorAll('.item');

// Todos los enlaces dentro de un nav
let enlaces = document.querySelectorAll('nav a');

// Elementos con múltiples condiciones
let inputs = document.querySelectorAll('input[required]');

// Recorrer con forEach (NodeList lo soporta)
items.forEach(function(item) {
    console.log(item.textContent);
});
```



Selectores CSS útiles para querySelector

```
// Selectores básicos
document.querySelector('div');           // Primera etiqueta div
document.querySelector('#miId');         // Elemento con id="miId"
document.querySelector('.miClase');      // Primer elemento con class=

// Selectores de atributos
document.querySelector('[data-id]');     // Elemento con atributo data
document.querySelector('[type="email"]'); // Input con type="email"
document.querySelector('[class*="btn"]'); // Elemento cuya clase conti
```



```
// Selectores de jerarquía
document.querySelector('nav > ul'); // ul hijo directo de nav
document.querySelector('div p'); // p descendiente de div
document.querySelector('h2 + p'); // p inmediatamente después c

// Pseudoselectores
document.querySelector('li:first-child'); // Primer li
document.querySelector('li:last-child'); // Último li
document.querySelector('tr:nth-child(2)'); // Segunda fila
document.querySelector('input:checked'); // Input marcado
```

Navegación entre elementos

Propiedades de navegación

Una vez que tienes un elemento, puedes navegar a elementos relacionados:

```
let elemento = document.querySelector('.mi-elemento');

// Navegación por nodos (incluye texto y comentarios)
console.log(elemento.parentNode); // Nodo padre
console.log(elemento.childNodes); // Todos los nodos hijos
console.log(elemento.firstChild); // Primer nodo hijo
console.log(elemento.lastChild); // Último nodo hijo
console.log(elemento.nextSibling); // Siguiete hermano
console.log(elemento.previousSibling); // Hermano anterior

// Navegación solo por elementos (ignora texto/comentarios)
console.log(elemento.parentElement); // Elemento padre
console.log(elemento.children); // Solo elementos hijos
console.log(elemento.firstElementChild); // Primer elemento hijo
console.log(elemento.lastElementChild); // Último elemento hijo
console.log(elemento.nextElementSibling); // Siguiete elemento he
console.log(elemento.previousElementSibling); // Elemento hermano ante
```

Ejemplo práctico de navegación

```
<div class="contenedor">
  <h2>Título de sección</h2>
  <p>Primer párrafo</p>
  <p class="destacado">Segundo párrafo</p>
  <ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
  </ul>
</div>
```

```
// Partimos del párrafo destacado
let parrafoDestacado = document.querySelector('.destacado');

// Obtener elementos relacionados
let contenedor = parrafoDestacado.parentElement;
let titulo = parrafoDestacado.previousElementSibling.previousElementSi
let lista = parrafoDestacado.nextElementSibling;
let primerItem = lista.firstElementChild;

console.log(contenedor.className); // "contenedor"
console.log(titulo.textContent);    // "Título de sección"
console.log(primerItem.textContent); // "Item 1"
```

Acceso y modificación de contenido

innerHTML vs textContent vs innerText

innerHTML

Obtiene o establece el HTML completo dentro de un elemento.

```
let div = document.querySelector('#mi-div');

// Obtener HTML
console.log(div.innerHTML);
```

```
// Ejemplo: "<p>Hola <strong>mundo</strong></p>"

// Establecer HTML
div.innerHTML = '<h2>Nuevo título</h2><p>Nuevo párrafo</p>';

// Añadir HTML al existente
div.innerHTML += '<p>Párrafo adicional</p>';
```

⚠ Cuidado con innerHTML:

- Puede ser vulnerable a ataques XSS si usas contenido no confiable
- Reemplaza todo el contenido existente
- Es más lento que textContent

textContent

Obtiene o establece solo el texto, ignorando las etiquetas HTML.

```
let div = document.querySelector('#mi-div');

// Obtener texto (sin HTML)
console.log(div.textContent);
// Ejemplo: "Hola mundo" (sin las etiquetas)

// Establecer texto (HTML será escapado)
div.textContent = 'Texto simple <strong>esto no será negrita</strong>'
```

innerText

Similar a textContent pero respeta los estilos CSS (elementos ocultos).

```
let div = document.querySelector('#mi-div');

console.log(div.innerText); // Solo texto visible
console.log(div.textContent); // Todo el texto, incluso el oculto
```

Modificación de atributos

getAttribute() y setAttribute()

```
let imagen = document.querySelector('img');

// Obtener atributos
let src = imagen.getAttribute('src');
let alt = imagen.getAttribute('alt');

// Establecer atributos
imagen.setAttribute('src', 'nueva-imagen.jpg');
imagen.setAttribute('alt', 'Nueva descripción');
imagen.setAttribute('data-id', '123');

// Verificar si tiene un atributo
if (imagen.hasAttribute('title')) {
    console.log('La imagen tiene título');
}

// Eliminar atributo
imagen.removeAttribute('title');
```



Propiedades directas

Muchos atributos HTML comunes se pueden acceder como propiedades:

```
let enlace = document.querySelector('a');

// Acceso directo (más rápido)
enlace.href = 'https://ejemplo.com';
enlace.target = '_blank';
enlace.title = 'Enlace externo';

let input = document.querySelector('input');
input.value = 'Nuevo valor';
input.placeholder = 'Escribe aquí';
input.disabled = true;
```



Trabajo con clases CSS



```
let elemento = document.querySelector('.mi-elemento');

// Propiedades de clase
console.log(elemento.className); // String con todas las clases
elemento.className = 'nueva-clase otra-clase';

// classList (más moderno y útil)
elemento.classList.add('nueva-clase'); // Añadir clase
elemento.classList.remove('vieja-clase'); // Quitar clase
elemento.classList.toggle('activo'); // Alternar clase
elemento.classList.contains('activo'); // Verificar si tiene clase

// Reemplazar clase
elemento.classList.replace('clase-vieja', 'clase-nueva');
```

Creación de elementos

createElement()

```
// Crear un nuevo elemento
let nuevoParrafo = document.createElement('p');

// Añadir contenido
nuevoParrafo.textContent = 'Este es un párrafo creado dinámicamente';

// Añadir atributos
nuevoParrafo.setAttribute('class', 'dinamico');
nuevoParrafo.id = 'parrafo-1';

// El elemento existe pero aún no está en el DOM
console.log(nuevoParrafo); // <p class="dinamico" id="parrafo-1">...</p>
```

Insertar elementos en el DOM

appendChild()

Añade un elemento al final de los hijos de otro elemento.


```
let contenedor = document.querySelector('#contenedor');
let nuevoDiv = document.createElement('div');
nuevoDiv.textContent = 'Nuevo contenido';

// Añadir al final
contenedor.appendChild(nuevoDiv);
```



insertBefore()

Inserta un elemento antes de otro elemento específico.

```
let contenedor = document.querySelector('#contenedor');
let referencia = document.querySelector('#elemento-existente');
let nuevoElemento = document.createElement('p');

contenedor.insertBefore(nuevoElemento, referencia);
```



insertAdjacentHTML()

Método más flexible para insertar HTML en posiciones específicas.

```
let elemento = document.querySelector('#mi-elemento');

// Antes del elemento
elemento.insertAdjacentHTML('beforebegin', '<p>Antes del elemento</p>')

// Al inicio del contenido
elemento.insertAdjacentHTML('afterbegin', '<span>Al inicio</span>');

// Al final del contenido
elemento.insertAdjacentHTML('beforeend', '<span>Al final</span>');

// Después del elemento
elemento.insertAdjacentHTML('afterend', '<p>Después del elemento</p>')
```



Ejemplo completo: Crear una lista dinámica



```
function crearLista() {
  // Crear el contenedor ul
  let lista = document.createElement('ul');
  lista.className = 'lista-dinamica';

  // Array de elementos
  let elementos = ['Elemento 1', 'Elemento 2', 'Elemento 3'];

  // Crear cada li
  elementos.forEach(function(texto, indice) {
    let item = document.createElement('li');
    item.textContent = texto;
    item.setAttribute('data-indice', indice);

    // Añadir al ul
    lista.appendChild(item);
  });

  // Añadir la lista completa al body
  document.body.appendChild(lista);
}

crearLista();
```

Eliminación de elementos

removeChild()

Método tradicional que requiere acceso al elemento padre.

```
let elemento = document.querySelector('#elemento-a-eliminar');
let padre = elemento.parentNode;
padre.removeChild(elemento);
```



remove() (moderno)

Método más simple y directo.

```
let elemento = document.querySelector('#elemento-a-eliminar');  
elemento.remove(); // Más simple y claro
```



Limpiar contenido

```
let contenedor = document.querySelector('#contenedor');  
  
// Eliminar todo el contenido HTML  
contenedor.innerHTML = '';  
  
// Eliminar solo el texto  
contenedor.textContent = '';  
  
// Eliminar todos los hijos uno a uno  
while (contenedor.firstChild) {  
    contenedor.removeChild(contenedor.firstChild);  
}
```



Clonación de elementos

cloneNode()

```
let original = document.querySelector('#elemento-original');  
  
// Clon superficial (solo el elemento, sin hijos)  
let clonSuperficial = original.cloneNode(false);  
  
// Clon profundo (elemento con todos sus hijos)  
let clonProfundo = original.cloneNode(true);  
  
// Modificar el clon antes de insertarlo  
clonProfundo.id = 'elemento-clonado';  
clonProfundo.querySelector('.titulo').textContent = 'Copia del original';  
  
// Insertar el clon  
document.body.appendChild(clonProfundo);
```



Ejemplo práctico: Lista de tareas

```
<!DOCTYPE html>
<html>
<head>
  <title>Lista de Tareas</title>
  <style>
    .completada {
      text-decoration: line-through;
      opacity: 0.6;
    }
    .tarea {
      margin: 5px 0;
      padding: 10px;
      border: 1px solid #ddd;
    }
  </style>
</head>
<body>
  <h1>Mi Lista de Tareas</h1>

  <div id="formulario">
    <input type="text" id="nueva-tarea" placeholder="Escribir nueva tarea">
    <button onclick="agregarTarea()">Agregar</button>
  </div>

  <div id="lista-tareas"></div>

  <script>
    let contadorTareas = 0;

    function agregarTarea() {
      let input = document.getElementById('nueva-tarea');
      let texto = input.value.trim();

      if (texto === '') {
        alert('Por favor escribe una tarea');
        return;
      }

      // Crear contenedor de la tarea
      let divTarea = document.createElement('div');
```

```

    divTarea.className = 'tarea';
    divTarea.id = 'tarea-' + contadorTareas;

    // Crear checkbox
    let checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.onChange = function() {
        marcarCompletada(this);
    };

    // Crear texto de la tarea
    let spanTexto = document.createElement('span');
    spanTexto.textContent = texto;
    spanTexto.style.marginLeft = '10px';

    // Crear botón eliminar
    let botonEliminar = document.createElement('button');
    botonEliminar.textContent = 'Eliminar';
    botonEliminar.style.marginLeft = '10px';
    botonEliminar.onclick = function() {
        eliminarTarea(divTarea.id);
    };

    // Ensamblar la tarea
    divTarea.appendChild(checkbox);
    divTarea.appendChild(spanTexto);
    divTarea.appendChild(botonEliminar);

    // Añadir al contenedor principal
    document.getElementById('lista-tareas').appendChild(divTar

    // Limpiar input y incrementar contador
    input.value = '';
    contadorTareas++;
}

function marcarCompletada(checkbox) {
    let tarea = checkbox.parentElement;
    if (checkbox.checked) {
        tarea.classList.add('completada');
    } else {
        tarea.classList.remove('completada');
    }
}

```

```

function eliminarTarea(id) {
    let tarea = document.getElementById(id);
    if (confirm('¿Estás seguro de que quieres eliminar esta ta
        tarea.remove();
    }
}

// Permitir agregar tarea con Enter
document.getElementById('nueva-tarea').addEventListener('keypr
    if (e.key === 'Enter') {
        agregarTarea();
    }
});
</script>
</body>
</html>

```

Buenas prácticas

1. Verificar la existencia de elementos

```

//  Buena práctica
let elemento = document.querySelector('#mi-elemento');
if (elemento) {
    elemento.textContent = 'Nuevo contenido';
} else {
    console.warn('Elemento no encontrado');
}

```



2. Cachear elementos frecuentemente usados

```

//  Malo: buscar cada vez
function actualizarContador() {
    document.getElementById('contador').textContent = '0';
    document.getElementById('contador').style.color = 'red';
}

```



```
//  Bueno: cachear elemento
let contador = document.getElementById('contador');
function actualizarContador() {
    contador.textContent = '0';
    contador.style.color = 'red';
}
```

3. Usar innerHTML con precaución

```
//  Peligroso con contenido no confiable
let userInput = getUserInput(); // Podría contener <script>alert('XSS')
div.innerHTML = userInput;

//  Seguro
div.textContent = userInput; // El HTML será escapado automáticamente
```

4. Agrupar modificaciones del DOM

```
//  Ineficiente: múltiples modificaciones
for (let i = 0; i < 1000; i++) {
    let p = document.createElement('p');
    p.textContent = 'Párrafo ' + i;
    document.body.appendChild(p); // Modifica DOM 1000 veces
}

//  Eficiente: crear fragmento
let fragmento = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
    let p = document.createElement('p');
    p.textContent = 'Párrafo ' + i;
    fragmento.appendChild(p);
}
document.body.appendChild(fragmento); // Modifica DOM 1 sola vez
```

5. Usar métodos modernos cuando sea posible

//  *Antiguo*

```
let elementos = document.getElementsByClassName('item');  
for (let i = 0; i < elementos.length; i++) {  
  elementos[i].style.color = 'red';  
}
```

//  *Moderno*

```
document.querySelectorAll('.item').forEach(elemento => {  
  elemento.style.color = 'red';  
});
```

Próximos pasos

Con el conocimiento de selección y manipulación del DOM, ahora puedes:

1. **Modificar estilos CSS** dinámicamente
2. **Manejar eventos** para crear interactividad
3. **Validar formularios** en tiempo real
4. **Crear interfaces dinámicas** complejas
5. **Trabajar con APIs** para contenido dinámico

La manipulación del DOM es fundamental para crear aplicaciones web interactivas. Practica estos conceptos creando pequeños proyectos antes de avanzar a temas más complejos.